PROGRAMMING IN C – UNIT 2

2 MARKS

1) What are operators? Specify their use.

An operator is a symbol that tells the computer to perform certain mathematical and logical manipulations.
Operator is used to manipulate data and variable.

2) List different categories of operators available in C.

1. Arithmetic operators
2. Relational operators
3. Logical operators
4. Assignment operators
5. Increment and decrement operators
6. Conditional operators
7. Bitwise operators
8. Special operators

3) What is integer arithmetic? Give an example.

When both the operands in the expression are integers then the operation is known as the integer arithmetic.
Example : a=4 , b=5 then a+b = 9
a=9, b=3 then a/b = 3 Here both a and b are integer values.

4) What is real arithmetic? Give an example.

An arithmetic operation involving only real operands is called as real arithmetic.
Example : a=2.2 , b=2.8 then a+b = 5. Here both a and b are real values.

5) What is relational expression? Give an example.

Relational operators are used to compare two quantities and to make decisions depending on their relation.
Example : < is less than
> is greater than

6) What is logical expression? Give an example.

An expression that combines more than one relational expression to make decision is called as a logical expression.
Example : && AND
|| OR
! NOT

7) Differentiate pre/post increment and decrement operators.

Increment operator (++) adds one to the operand. For example if a=2 then a++ = 3.
Decrement operator (- -) subtracts one from the operand. For example if a=2 the a-- = 1.

8) List short-hand assignment operators available in C.

Shorthand operators : += (a=a+1 equivalent to a+=1)
-= (a=a-1 equivalent to a-=1)
*= (a=a*2 equivalent to a*=2)
/= (a=a/6 equivalent to a/=6)
%= (a=a%1 equivalent to a%=1)

9) What is the difference between = and == operators?

= operator is used for assigning value to the variable.
= = operator is used for comparing two values. It returns 1 if both the values are equal else returns 0.

10) Give the syntax of conditional operator. Give an example.

Syntax : exp1 ? exp2 : exp3
Example : Let a=2 and b=4
x = (a>b) ? a :b;
If the condition is true then x will be assigned the value a else it will be assigned the value of b. In this x will be assigned the value of b since the condition is false.

11) List special operators available in C along with their meaning.

Comma operator – Used to link the related expression together.
sizeof operator - Used to determine the length of the operands.
& operator - Used to determine the address of the variable.
* (pointer operator) – Used to point to the address of the another variable.

12) Define arithmetic expression. Give an example.

Arithmetic expression is a combination of variables, constants and operators arranged as per the syntax of the language.
Example : x = a+c/b*a;
Here a=2, c=8, b=4 then
x = 2+8/4*2
= 2+2*2
=2+4 = 6

13) Specify the precedence of arithmetic operators in C.

An arithmetic expression without parenthesis will be evaluated from left to right using the rules of precedence of operators. There are two priority levels of arithmetic opertaors.
High priority * / %
Low priority + -

14) What do you mean by type conversion? Mention its types.

Type conversion is the process of changing an entity from one data type to another.
There are two types: 1. Implicit Type Conversion
2. Explicit Type Conversion

15) List any four built-in Mathematical functions available in C.

sin(x) - Sine of
cos(x) - Cosine of x
tan(x) - Tangent of x
atan(x)- Arc tangent of x

16) Write the syntax of simple if statement. Give an example.

Syntax : if(expression)
{
Statement;
}

```
Example :  if(avg<35)
           {
                printf("Fail")
           }
```

17) Give the syntax of goto statement. Give an example.

```
Syntax : goto label;
         ……
         ……..
         label;
         statement;

Example :  main()
           {
            int x;
            read:
            scanf("%d", &x);
            if(x<0)
            goto read;
            }
```

18) Define the term looping. What are the different looping statements available in C?

Looping is the process where the sequence of statements are continuously executed until a specific condition is met.
Different looping statements available in C are :

1. while loop
2. do while loop
3. for loop

19) Differentiate entry-controlled and exit-controlled loops.

while is an entry controlled loop statement. In while loop the condition is tested before the execution of the loop. Therefore the body of the loop may not be executed at all if the condition is false at the very first attempt.

do while is an exit controlled loop. do while loop the condition is tested after the execution of the loop. Therefore the body of the loop is executed atleast once.

20) Differentiate break and continue statements.

Break statement is used to get the control of the program out of the loop. When a break statement is encountered inside a loop, the loop is immediately exited and the program continuous with the statement immediately

Continue statement is used to get the control of the program to the beginning of the loop. Continue statement tells the compiler to skip the following statements and continue with the next iteration.

6 MARKS

1) List and explain arithmetic, logical and relational operators available in C

Operator is a symbol that tells the computer to perform certain mathematical or logical operations. Operators are used in programs to manipulate data and variables.

Arithmetic operators: Arithmetic operators are mathematical functions that take two operands and perform calculations on them.

Basic arithmetic operators are

Addition	+
Subtraction	-
Multiplication*	
Division	/
Modulo division	%

Let a=10 and b=5

a + b = 15
a – b = 5
a * b = 50
a / b = 2
a % b = 0

Its further divided into : Integer arithmetic , Real arithmetic and Mixed mode arithmetic

Relational operators : Relational operators are used to compare two quantities and to make decisions depending on their relation.

There are 6 relational operators:

<	is less than
>	is greater than
<=	is less than or equal to
>=	is greater than equal to
==	is equal to
!=	is not equal to

Let a=10 and b=5

a < b	-> 10 < 5	FALSE
a > b	-> 10 > 5	TRUE
a <= b	-> 10 <= 5	FALSE
a >= b	-> 10 <= 5	TRUE
a == b	-> 10 == 5	FALSE
a != b	-> 10 != 5	TRUE

The value of the relational expression cab either be TRUE or FALSE.

Logical operators : Logical operators are used when we want to test more than one condition and make decision.

There are 3 logical operators

&&	AND
||	OR
!	NOT

Let a=10 and b=5

a>b && a!=b
TRUE && TRUE -> TRUE

a==b || a>b
FALSE || TRUE -> FALSE

a	b	a && b	a || b
0 (FALSE)	0 (FALSE)	0 (FALSE)	0 (FALSE)
0 (FASLE)	1 (TRUE)	0 (FASLE)	1 (TRUE)
1 (TRUE)	0 (FASLE)	0 (FASLE)	1 (TRUE)
1 (TRUE)	1 (TRUE)	1 (TRUE)	1 (TRUE)

2) List and explain assignment, increment and decrement operators available in C.

Assignment operators : Assignment operators are used to assign the result of an expression to a variable. The usual assignment operator is '='. C has a set of shorthand operators of the form

v op = exp;

where v is the variable, exp is an expression and op is an arithmetic operator. The operator op= is known as the shorthand operator.

The assignment statement

v op = exp;

equivalent to

v = v op(exp)

Example :

x +=y;	is same as	x = x+y;
x -=y+1;	is same as	x = x – (y+1)
x *= 4;	is same as	x = x*4;;

The use of shorthand assignment operators has three advantages :

1. What appears on the left hand side need not be repeated and therefore it becomes easier to write.
2. The statement is easier to read.
3. The statement is more efficient.

Increment and decrement operators :

The increment operator is ++ and the decrement operator is –

The operator ++ adds 1 to the operand, and -- subtracts 1. Both takes the following form:

++m; or m++;

--m; or m--;

We use the increment and decrement statements in for and while loops.

While ++m and m++ , --m and m-- mean the same when they form statements independently. But they behave differently when they are used in expressions on the right hand side of an assignment statement.

Example :

Consider m = 5;

y = ++m;

In this case the value of y and m would be same 6. If we write the above statement as

m = 5;

y = m++;

then, the value of y would be 5 and m would be 6.

A prefix operator first adds 1 to the operand and then the result is assigned to the variable on left. On the other hand, a postfix operator first assigns the value to the variable on left and then increments the operand.

3) List and explain conditional, bitwise and special operators available in C.

Conditional operators : Conditional operator is also called as the ternary operator. Ternary operator pair is " ?: "

The conditional operator form is : exp1 ? exp2 : exp3

exp1 is evaluated first. If it is nonzero(true),then the expression exp2 is evaluated and becomes the value of the expression. If exp1 is false, exp3 is evaluated and its value becomes the value of the expression. Only one of the expressions(either exp2 or exp3) is evaluated.

For example,

a = 10;

b = 15;

x = (a>b) ? a:b

Here x will be assigned the value of b. This can be achieved using if else statement

```
if(a>b)
x=a;
else
x=b;
```

Bitwise Operators:

C has a distinction of supporting special operators known as bitwise operators for manipulation of data at bit level.

&	bitwise AND
|	bitwise OR
^	bitwise exclusive OR
<<	shift left
>>	shift right

Ex: 1001101001
<<(shift left) x=3
1101001000

1001 & 1110 = 1000
1010 | 1001 = 1011

Special Operators:

Special operators are comma operator and sizeof operator.

Comma operator : The comma operator can be used to link the related expressions together. A comma linked list of expressions are evaluated left to right and the value of right most expression is the value of the combined expression.
Ex: value(x=10,y=5,x+y)
First assigns the value 10 to x, then assigns 5 to y, and finally assigns 15(i.e 10+5) to value.

The sizeof operator: The sizeof is a compile time operator and when used with an operand it returns the number of bytes the operand occupies.
Ex : m=sizeof(sum)
n=size(long int)

The sizeof operator is normally used to determine the lengths of arrays and structures when their sizes are not known to the programmer. It is also used to allocate memory space dynamically to variables during execution of a program.

4) Explain with Example evaluation of arithmetic expression in c.

- An arithmetic expression is a combination of variables, constants, and operators arranged as per the syntax of the language**.**

EVALUATION OF EXPRESSIONS

Expressions are evaluated using an assignment statement of the form:

variable = expression;

Variable is any valid C variable name. When the statement is encountered, the expression is evaluated first and the result then replaces the previous value of the variable on the left-hand side. All variables used in the expression must be assigned values before evaluation is attempted. Examples of evaluation statements are

x = a * b - c;
y = b / c * a;
z=a – b / c + d;
The blank space around an operator is optional and adds only to improve readability. When these statements are used in a program, the variables a, b, c, and d must be defined before they are used in the expressions.

Example:
Program

```
main()

{
        float a, b, c, x, y, z;
        a = 9;
        b = 12;
        c = 3;
x = a - b / 3 + c * 2 - 1;
y = a - b / (3 + c) * (2 - 1);
z = a - (b / (3 + c) * 2) - 1;
printf("x = %f", x);
printf("y = %f", y);
printf("z = %f", z);
}
```

OUTPUT:
x = 10.000000
y = 7.000000
z = 4.000000

5) Explain precedence of arithmetic operators with the help of an example expression

An arithmetic expression without parentheses will be evaluated from left to right using the rules of precedence of operators. There are two distinct priority levels of arithmetic operators in C:

High priority * / %

Low priority + -

The basic evaluation procedure includes two' left-to-right passes through the expression. During the first pass, the high priority operators (if any) are applied as they are encountered. During the second pass, the low priority operators (if any) are applied as they are encountered.

Let us consider and evaluation for example

x = a-b/3 + c*2-1

When a = 9, b = 12, and c = 3, the statement becomes.

x = 9-12/3+3*2-1

and is evaluated as follows

First pass
Step1: x=9-4+3*2-1

Step2: x=9-4+6-1

Second pass

Step3: x =5+6-1

Step4: x = 11-1

Step5: x = 10

The order of evaluation can be changed by introducing parentheses into an expression. Consider the same expression with parentheses as shown below:

9-12/(3+3)*(2-1)

Whenever parentheses are used, the expressions within parentheses assume highest priority. If two or more sets of parentheses appear one after another as shown above, the expression contained in the left-most set is evaluated first and the right-most in the last.

For example:

9-(12/(3+3) *2)-1=4

Where as

9-((12/3) + 3 * 2)-1 = -2

While parentheses allow us to change the order of priority, we may also use them to improve understandability of the program.

Rules for Evaluation of Expression:

- First, parenthesized sub expression from left to right are evaluated.
- If parentheses are nested, the evaluation begins with the innermost sub-expression.
- The precedence rule is applied in determining the order of application of operators in evaluating sub expressions.
- The associativity rule is applied when two or more operators of the same precedence level appear in a sub-expression.
- Arithmetic expressions are evaluated from left to right using the rules of precedence.
- When parentheses are used, the expressions within parentheses assume highest priority.

6) Explain the concept of type conversion in C, Give examples.

Implicit type conversion:

C automatically converts any intermediate values to the proper type so that the expression can be evaluated without losing any significance. This automatic conversion is known as implicit type conversion.

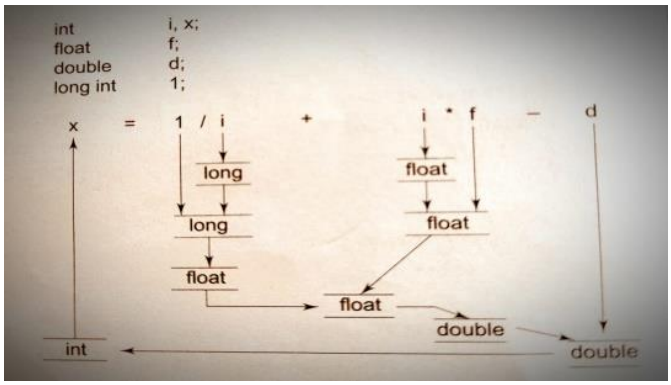
- C permits mixing of constants and variables of different types in an expression.

- During evaluation it follows very strict rules of type conversion. If the operands are of different types, the lower type is automatically converted to the higher type before the operation proceeds. The result is of the higher type.

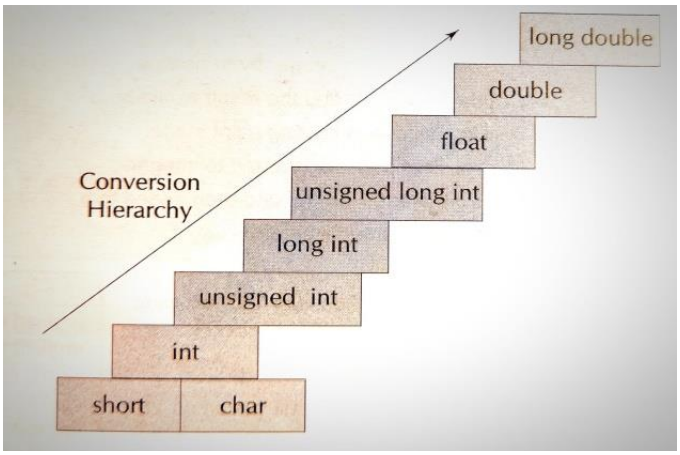
Given below is the sequence of rules that are applied while evaluating expressions.

- All short and char are automatically converted to int
- If one of the operands is double, the other will be converted to double and the result will be double;
- If one of the operands is float, the other will be converted to float and the result will be float;
- If one of the operands is long int, the other will be converted to long int and the result will be long int;
- If one of the operands is long int and the other is unsigned int, then the result will be long int;
- If one of the operands is long int, the other will be converted to long int and the result will be long int;

Example:



Conversion Hierarchy



<u>Explicit Conversion</u>

C performs type conversion automatically through implicit type conversion. However, there are instances when we want to force a type conversion in a way that is different from the automatic conversion. Consider, for example, the calculation of ratio of females to males in a town.

ratio =female_ number/male_number

Since female_number and male_number are declared as integers in the program, the decimal part of the result of the division would be lost and ratio would represent a wrong figure. This problem can be solved by converting locally one of the variables to the floating point as shown below:

ratio = (float) female_number/male_number

The operator (float) converts the female_number to floating point for the purpose of evaluation of the expression. Then using the rule of automatic conversion, the division is performed in floating point mode, thus retaining the fractional part of result.

The process of such a local conversion is known as explicit conversion Or casting a value. The general form of casting is

(type-name) expression

where type-name is one of the standard C data types. The expression may be a constant, variable or an expression. Some examples of casts and their actions are shown in below

Casting can be used to round-off a given value. Consider the following statement:

X =(int) (y+0.5);

If y is 27.6, y+0.5 is 28.1 and on casting, the result becomes 28, the value that is assigned to x. Of course, the expression, being cast is not changed.

7) Explain in brief implicit and explicit type conversion with example.

Type Conversion

The process of converting the value of one data type (integer, string, float, etc.) to another data type is called type conversion.

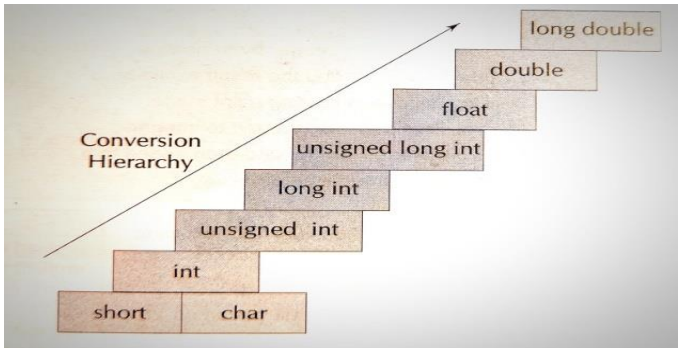
- There are two types in Type Conversion
- **Implicit Type Conversion**
- **Explicit Type Conversion**

C automatically converts any intermediate values to the proper type so that the expression can be evaluated without losing any significance. This automatic conversion is known as implicit type conversion.

- C permits mixing of constants and variables of different types in an expression.
- During evaluation it follows very strict rules of type conversion. If the operands are of different types, the lower type is automatically converted to the higher type before the operation proceeds. The result is of the higher type.

Given below is the sequence of rules that are applied while evaluating expressions.

- All short and char are automatically converted to int
- If one of the operands is double, the other will be converted to double and the result will be double;
- If one of the operands is float, the other will be converted to float and the result will be float;
- If one of the operands is long int, the other will be converted to long int and the result will be long int;
- If one of the operands is long int and the other is unsigned int, then the result will be long int;



Explicit Conversion

C performs type conversion automatically through implicit type conversion. However, there are instances when we want to force a type conversion in a way that is different from the automatic conversion. Consider, for example, the calculation of ratio of females to males in a town.

ratio =female_ number/male_number

Since female_number and male_number are declared as integers in the program, the decimal part of the result of the division would be lost and ratio would represent a wrong figure.

This problem can be solved by converting locally one of the variables to the floating point as shown below:

ratio = (float) female_number/male_number

The operator (float) converts the female_number to floating point for the purpose of evaluation of the expression. Then using the rule of automatic conversion, the division is performed in floating point mode, thus retaining the fractional part of result.

8) Explain different forms of if statement.

If statement is a decision making statement used to control the flow of execution of the statements.
The different forms of if statements are :

a) Simple if statement
b) if else statement
c) Nested if else statement
d) else if ladder

a) **simple if statement** : When there is only one condition then simple if statement is used. If the test condition is true the statement block will be executed otherwise the statement will be skipped and the control of the program goes to the next statement in the program.

Syntax :

```
if(expression)
{
statement;
}
statement x;
```

Example :

```
If(a==b)
{
printf("a and b are equal");
}
```

b) **if else statement** : When there are two conditions then if else statement is used. If the condition or the expression is true then the first statement is executed i.e the true block statements otherwise the else statement is executed i.e the false block statement. Either first statement or the second statement will be executed, not both.

Syntax (General form) :

```
if(expression)
{
true statement ;
{
else
```

```
{
false statement ;
}
```

Example :

```
if(age>18)
{
printf("Person is eligible to vote");
}
else
{
printf("Person is not eligible to vote");
}
```

c) **nested if else statement** : When there are series of decisions involved then we have to use more than one if else statement in nested form.

Syntax (General form) :

```
if(condition 1)
{
if(condition 2)
{
statement 1;
}
else
{
statement 2;
}
}
else
{
statement 3;
}
Statement x;
```

If the condition 1 is false then statement 3 is executed. If condition 1 is true then condition 2 is tested. If both the conditions are true then statement 1 is executed, otherwise the statement 2 will be executed.

Example :

```
if(n1>n2)
{
if(n1>n3)
printf("n1 is the largest");
else
printf("n3 is the largest");
}
else
{
if(n2>n3)
printf("n2 is the largest");
else
printf("n3 is the largest");
}
```

d) **else if ladder**: When there are more than two conditions to be tested then else if is used. Here the conditions are tested from top to bottom. If the first condition is true statement 1 is executed else second statement is tested and if its true then the statement 2 will be executed. If the second condition is also false then the third condition is tested and so on. Any one statement is executed which is associated with the true condition. If none of the condition is true then the default statement is executed.

Syntax (General form) :

```
if(condition 1)
{
statement 1;
}
else if(condition 2)
{
statement 2;
}
else if(condition 3)

{
statement 3;
}
.
.
.
else if(condition n)
{
Statement n;
}
else
{
default statement;
}
Statement x;
```

Example :

```
if(avg>90)
{
printf("Grade is A");
}
else if(avg>70)
{
printf("Grade is B");
}
else if(avg>50)
{
printf("Grade is C");
}
else if(avg>35)
{

printf("Grade is D");
}
else
{
printf("FAIL");
}
```

9) Explain nested if statement with the help of a program example

When there are series of decisions involved then we have to use more than one if else statement in nested form that is using the nested if statement.

Syntax (General form) :

```
if(condition 1)
{
if(condition 2)
{
statement 1;
}
else
{
statement 2;
}
}
else
{
statement 3;
}
Statement x;
```

If the condition 1 is false then statement 3 is executed. If condition 1 is true then condition 2 is tested. If both the conditions are true then statement 1 is executed, else the statement 2 will be executed.

Example :

```
#include<stdio.h>
main()
{
int a,b,c;
clrscr();
printf("Enter the numbers:");
scanf("%d%d %d",&a,&b,&c);
if(a>=b)
{
if(a>=c)
printf("%d is the largest number",a);
else
printf("%d is the largest number",c);
}
else
{
if(b>=c)
printf("%d is the largest number",b);
else
printf("%d is the largest number",c);
}
getch();
return 0;
}
```

This program is to find the largest of three numbers using nested if statement. Three integer values are stored inside the variables a b and c. is tested. If both these conditions are true then the value of a will be printed. then the condition is tested for a>b if the condition is true again the condition a>c .If the first condition a>b is false then the control will go to the else statement and condition b > c is tested. If this is true the value of b is printed else the value of c will be printed.

10) Explain else..if ladder with the help of a program example.

When there are more than two conditions to be tested then else if is used. Here the conditions are tested from top to bottom. If the first condition is true statement 1 is executed else second statement is tested and if its true then the statement 2 will be executed. If the second condition is also false then the third condition is tested and so on. Any one statement is executed which is associated with the true condition. If none of the condition is true then the default statement is executed.

Example :

```
#include<stdio.h>
main()
{
int m1,m2,m3,m4,m5,total;
float per;
clrscr();
printf("Enter the marks in five subjects:\n");
scanf("%d%d%d%d%d",&m1,&m2,&m3,&m4,&m5);
total=m1+m2+m3+m4+m5;
per=total/5;
if(per>=60 && per<=100)
printf("Grade = A");
else if(per>=50 && per<=60)
printf("Grade = B");
else if(per>=40 && per<=50)
printf("Grade = C");
else
printf("FAIL");
getch();
return 0;
}
```

This is a program to find the grade of students. Marks of five subjects are entered through the keyboard and stored in the variables m1,m2,m3,m4 and m5. The total and average is found using the formulas. Then the else if ladder is used to find the grade of the student. If the first condition is true the first statement is printed, else the second condition is tested. If it is true the second statement is executed else the third statement is tested and so on. The process continues until the value matches and the grade associated with it will be executed. If none of the values match then the default statement 'FAIL' will be printed.

11) Explain Switch statement with Syntax and Example.

C has a built-in multiway decision statement known as switch. The switch statement tests the value of a given expression against a list of case values and when a match is found, a block of statements associated with that case is executed.

Syntax :

```
switch(expression)
{
case value 1 :
                block 1;
                break;
case value 2 :  block 2;
                break;

……
……
default :       default block;
                break;

}
```

Here expression is an integer expression or characters. value-1, value-2……are considered as the case labels. Each of these values should be unique within the switch statement. block-1, block-2…..are the statement lists which contain zero or more statements. When switch is executed, the expression is compared with the values value-1,value-2….. If a case is found whose value matches with the value of the expression then the block of statements of that case will be executed. break statement is used to end each block that causes exit from switch statement. The default block is executed if the value of the expression does not match with any of the case values.

Ex**ample:**

```
ch=getchar();
switch(ch)
{
case '1':
        printf("Addition of two integer is:%d",a+b);
        break;
case '2':
        printf("Subtraction of two integer is :%d",a-b);
        break;
default:
        printf("Invalid choice");
}
```

Here the value of the variable ch will be compared with case values 1 and 2. If the value of variable ch is 1 then the block followed by case 1 will be executed. If the value of ch is 2 the block of case 2 is executed. If the value of ch does not match with both of these cases then the default block is executed.

12) Explain while loop with syntax and example.

The simplest of all the looping statements in C is the while statement.

While loop is used to continuously execute the instructions for unknown number of times.

Syntax:

```
while(test condition)
{
//body of the loop
}
```

The while is an entry controlled loop statement.

Firstly the condition is tested, if the condition is true the body of the loop is executed. Then the condition will be evaluated once again and if it is true, the body is executed once again. This process continuous until the condition becomes false. Then the control is transferred out of the loop. In while loop the condition is tested before the execution of the loop. Therefore the body of the loop may not be executed at all if the condition is false at the very first attempt.

Example:

```
int i=1;
while(i<5)
{
    printf("%d",i);
    i++;
}
```

Here i is the integer variable which has been assigned with the initial value 1.while statement checks for the condition i<5 if this condition is true then the statement within the loop will be executed that is the value of i will be printed

which is 1. Then the value of i will be incremented by 1 so the value of i becomes 2. The condition i<5 is once again tested and if it is true the value of i is again printed. This process continuous until the condition becomes false, once the condition is false the loop will be terminated. So the final output will be 1 2 3 4.

13) Explain do while loop with syntax and example.

do while loop is used to continuously execute the instructions. do while loop executes a block of code atleast once.

Syntax :

```
do
{
  //body of the loop
}
while(condition);
```

do while is an exit controlled loop.
Here the body of the loop is executed first, at the end of the loop the test condition in the while statement is evaluated. If the condition is true, the body of the loop is executed once again. The process continues until the condition becomes false and the loop will be terminated and the control will go to the next statement in the program. Unlike while loop in do while loop the condition is tested after the execution of the loop. Therefore the body of the loop is executed atleast once.

Example:

```
int i=1;
do
{
        printf("%d",i);
        i++;
}
        while(i<5);
```

Here i is the integer variable which has been assigned with the initial value 1. The body of the loop is executed first that is the value of i will be printed which is 1. Then the value of i will be incremented by 1 so the value of i becomes 2. After that the while statement checks for the condition i<5, if this condition is true then the body of the loop executed once again. The condition i<5 is once again tested and if it is true the value of i is again printed. This process continuous until the condition becomes false, once the condition is false the loop will be terminated. So the final output will be 1 2 3 4. Since the condition is tested at the end of the loop the initial value of i will be printed even if the condition is false at the first evaluation.

14) Explain For loop with syntax and example.

for loop is used to continuously execute the instructions for definite number of times.

Syntax:

```
for(initialization;condition; increment)
{
      //body of the loop
}
```

- In for statement initialization of variables is done first, using assignment statement such as i=0.
- The value of the variable is tested using the test condition. The test condition is relational expression such as i<5 that determines when the loop will exit. If the condition is true then the body of the loop is executed if false then the loop will be terminated.
- When the body of the loop will be executed the control is transferred back to the for statement. Then the variable will be incremented by 1 using the assignment statement i=i+1. Then the value of the variable

is again tested, if it true the body of the loop is again executed. This process continuous until the condition becomes false. Once the condition is false the loop will be terminated and the control goes to the next statement in the program.

Example:

```
for(i=0; i<5; i++)
{
   printf ("%d",i);
}
```

Here the initial value of variable i is 0. The condition here is i<5 which will be tested, if the condition is true for 0<5 then the statement will be printed that is the value of i will be printed which is 0. Then the control will go back to the increment section of for loop and the value of i will be incremented by i so the value of i becomes 1. Then again the condition is tested for 1<5, if this is true then again the value of i will be printed. This will continue until the condition becomes false. Once the condition is false loop will be terminated and the control will go to the next statement in the program.

15) Explain the following.

i. Nested loops

ii. Infinite loops

Nested loops

C supports nesting of loops in C. **Nesting of loops** is the feature in C that allows the looping of statements inside another loop.

Any number of loops can be defined inside another loop, i.e., there is no restriction for defining any number of loops. The nesting level can be defined at n times. We can define any type of loop inside another loop, for example, we can define '**while**' loop inside a '**for**' loop.

Example:

```
int main()
{
        int n;
        printf("Enter the value of n :");
        for(int i=1;i<=n;i++)  // outer loop

      {
           for(int j=1;j<=10;j++)  // inner loop
           {
              printf("%d\t",(i*j)); // printing the value.
           }
       }
  }
```

First, the 'i' variable is initialized to 1 and then program control passes to the i<=n.
The program control checks whether the condition 'i<=n' is true or not.If the condition is true, then the program control passes to the inner loop. The inner loop will get executed until the condition is true. After the execution of the inner loop, the control moves back to the update of the outer loop, i.e., i++.
After incrementing the value of the loop counter, the condition is checked again, i.e., i<=n. If the condition is true, then the inner loop will be executed again. This process will continue until the condition of the outer loop is true.

Infinite loops

An infinite loop is a looping construct that does not terminate the loop and executes the loop forever. It is also called an indefinite loop or an endless loop. It either produces a continuous output or no output.

An infinite loop is useful for those applications that accept the user input and generate the output continuously until the user exits from the application manually. In the following situations, this type of loop can be used:

- All the operating systems run in an infinite loop as it does not exist after performing some task. It comes out of an infinite loop only when the user manually shuts down the system.
- All the servers run in an infinite loop as the server responds to all the client requests. It comes out of an indefinite loop only when the administrator shuts down the server manually.

- The following are the loop structures through which we will define the infinite loop:
  - for loop
  - while loop
  - do-while loop
  - go to statement
  - C macros

Example:

```
#include <stdio.h>
 main()
{
int i=0;
while(1)
{
i++;
printf("i is :%d",i);
}
}
```